SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4, BL5)
Assessment Tool Weightage: 25%

QUESTIONS: 05 **Total Marks:25**

Annexure A

Error Analysis Task

Code of Conduct:

I ______S.Poojitha(192320076)______________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Poojitha

RUBRICS FOR CALCULATION:

Error Analysis Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Annexure A

Error Analysis Task

Q1. Scenario: Online Banking System

Modules:
User Login, Fund Transfer, Account Verification

Question:

Write pseudocode for a quality assurance testing process that validates secure user authentication and fund transfer operations. Include input validation, transaction verification, exception handling, and test result logging.

Ans) START

```
Initialize Test Log

// 1. User Authentication Testing

INPUT username, password

IF username is empty OR password is empty THEN

    Log "Authentication failed: Missing input"

ELSE

    Validate username and password format

    IF format is invalid THEN

        Log "Authentication failed: Invalid input"

    ELSE

        Attempt secure login

        IF login is successful THEN

            Log "Authentication test PASSED"

        ELSE
```

```
            Log "Authentication test FAILED"
        END IF
    END IF
END IF
// 2. Account Verification
Verify authenticated user's account
IF account is valid AND active THEN
    Log "Account verification PASSED"
ELSE
    Log "Account verification FAILED"
    STOP
END IF
// 3. Fund Transfer Testing
INPUT receiver_account, amount
IF receiver_account is empty OR amount <= 0 THEN
    Log "Transfer failed: Invalid input"
ELSE IF receiver_account does not exist THEN
    Log "Transfer failed: Invalid receiver account"
ELSE IF amount > available_balance THEN
    Log "Transfer failed: Insufficient balance"
ELSE
    TRY
        Start transaction
```

```
        Verify sender and receiver accounts
        Deduct amount from sender account
        Add amount to receiver account
        Commit transaction
        IF transaction is successful THEN
            Log "Fund transfer test PASSED"
        ELSE
            Rollback transaction
            Log "Fund transfer test FAILED"
        END IF
    CATCH Exception
        Rollback transaction
        Log "Transfer error: Transaction rolled back"
    END TRY
END IF
// 4. Generate Test Report
Save all test results with:
    Test Case ID
    Input
    Expected Result
    Actual Result
    PASS/FAIL status
    Error details
```

END

Q2. Scenario: E-Commerce Shopping Platform

Features:
Product Search, Shopping Cart, Payment Gateway

Question:

Design pseudocode for a usability testing evaluation that measures customer interaction efficiency, including product search time, checkout completion time, and navigation complexity. Explain how the algorithm collects and analyzes user behavior data.

ANS) START

Initialize Test Results

FOR each test user

    // Product Search

    Record Search_Start_Time

    Ask user to search for a product

    Record Search_End_Time

    Search_Time = Search_End_Time - Search_Start_Time

    Store Search_Time

    // Navigation Complexity

    Set Click_Count = 0

    Count clicks and pages visited while user navigates

    Store Click_Count

    // Checkout

    Record Checkout_Start_Time

    Ask user to add product to cart and complete payment

```
    Record Checkout_End_Time
    Checkout_Time = Checkout_End_Time - Checkout_Start_Time
    Store Checkout_Time
    IF payment is successful THEN
        Store "Checkout Successful"
    ELSE
        Store "Checkout Failed"
    END IF
END FOR
// Analyze User Behavior
Calculate Average_Search_Time
Calculate Average_Checkout_Time
Calculate Average_Click_Count
Calculate Checkout_Success_Rate
IF Average_Search_Time is high THEN
    Recommend improving search and filters
END IF
IF Average_Checkout_Time is high THEN
    Recommend simplifying checkout
END IF
IF Average_Click_Count is high THEN
    Recommend simpler navigation
END IF
```

Generate Usability Report

END

Q3. Scenario: Hospital Management System

Modules:
Patient Registration, Appointment Scheduling, Medical Records

Question:

Write pseudocode to generate and execute test cases for appointment scheduling functionality. Include validation of doctor availability, appointment conflicts, patient information accuracy, and system responses to invalid requests.

Ans)START

```
Initialize Test Case List

Initialize Test Results

// Test Case 1: Valid Appointment

Enter valid patient information

Select doctor and appointment date/time

IF patient information is accurate AND

  doctor is available AND

  no appointment conflict exists THEN

   Book appointment

   Verify appointment confirmation

   Log "Test Case 1: PASSED"

ELSE

   Log "Test Case 1: FAILED"

END IF
```

```
// Test Case 2: Doctor Unavailable
Select a doctor with no available time slot
IF system rejects the appointment AND
   displays "Doctor not available" THEN
    Log "Test Case 2: PASSED"
ELSE
    Log "Test Case 2: FAILED"
END IF
// Test Case 3: Appointment Conflict
Use a time slot already booked by the patient
IF system prevents duplicate/conflicting booking THEN
    Log "Test Case 3: PASSED"
ELSE
    Log "Test Case 3: FAILED"
END IF
// Test Case 4: Invalid Patient Information
Enter missing or incorrect patient details
IF system rejects the request AND
   displays an appropriate error message THEN
    Log "Test Case 4: PASSED"
ELSE
    Log "Test Case 4: FAILED"
END IF
```

// Test Case 5: Invalid Appointment Request

Enter invalid date/time or past date

IF system rejects the request AND

shows a clear error message THEN

Log "Test Case 5: PASSED"

ELSE

Log "Test Case 5: FAILED"

END IF

// Generate Report

Display all test cases and their results

Calculate total PASSED and FAILED cases

END

Q4. Scenario: Airline Reservation System

Modules:
Passenger Registration, Flight Booking, Seat Allocation

Question:

Develop pseudocode for a constraint-based testing approach where booking requests must satisfy conditions such as valid passenger information, seat availability, and payment confirmation. Show how invalid booking scenarios are detected and handled.

Ans) START

Initialize Test Results

FOR each booking request

// Constraint 1: Validate Passenger Information

IF passenger name, ID, email, and contact number are valid THEN

```
    Passenger_Valid = TRUE
ELSE
    Passenger_Valid = FALSE
    Log "Invalid passenger information"
END IF
// Constraint 2: Check Seat Availability
IF requested seat is available THEN
    Seat_Available = TRUE
ELSE
    Seat_Available = FALSE
    Log "Seat not available"
END IF
// Constraint 3: Verify Payment
Process payment
IF payment is successful THEN
    Payment_Confirmed = TRUE
ELSE
    Payment_Confirmed = FALSE
    Log "Payment failed"
END IF
// Final Booking Validation
IF Passenger_Valid = TRUE AND
   Seat_Available = TRUE AND
```

Payment_Confirmed = TRUE THEN

Confirm booking

Allocate seat

Generate ticket

Log "Booking PASSED"

ELSE

Cancel booking

Release reserved seat

Display appropriate error message

Log "Booking FAILED"

END IF

END FOR

Generate Test Report

END

Q5. Scenario: Smart Library Management System

Features:
Book Search, Book Reservation, Borrow/Return Services

Question:

Write pseudocode for integrating quality assurance and usability testing where the system evaluates both functional correctness (reservation and borrowing operations) and user

Ans) START

Initialize Test Results

```
FOR each test user
    // 1. Book Search – Functional Testing
    Enter book title
    Record start time
    Search for book
    Record end time
    Response_Time = end time - start time
    IF correct book details are displayed THEN
        Log "Book Search PASSED"
    ELSE
        Log "Book Search FAILED"
    END IF
    // 2. Book Reservation – Functional Testing
    Select available book
    Click Reserve
    IF reservation is successfully created THEN
        Log "Reservation PASSED"
    ELSE
        Log "Reservation FAILED"
    END IF
    // 3. Borrow/Return – Functional Testing
    Select reserved book
    Click Borrow
```

```
IF borrowing is recorded correctly THEN
   Log "Borrow PASSED"
ELSE
   Log "Borrow FAILED"
END IF
Return book
IF return status is updated correctly THEN
   Log "Return PASSED"
ELSE
   Log "Return FAILED"
END IF
// 4. Usability Testing
Count clicks required to complete each task
Record navigation steps
Ask user to rate:
   Ease of navigation
   Readability
   Ease of completing tasks
Check accessibility:
   Font size
   Color contrast
   Keyboard navigation
   Screen-reader compatibility
```

```
END FOR

// 5. Analyze Results

Calculate:

    Average Response Time

    Average Navigation Steps

    Task Success Rate

    Average User Satisfaction Score

    Accessibility Issues

IF response time is high THEN

    Optimize search and database queries

END IF

IF navigation steps are high THEN

    Simplify menus and page layout

END IF

IF accessibility problems are found THEN

    Improve fonts, contrast, labels, and keyboard support

END IF

Generate QA and Usability Report

END
```